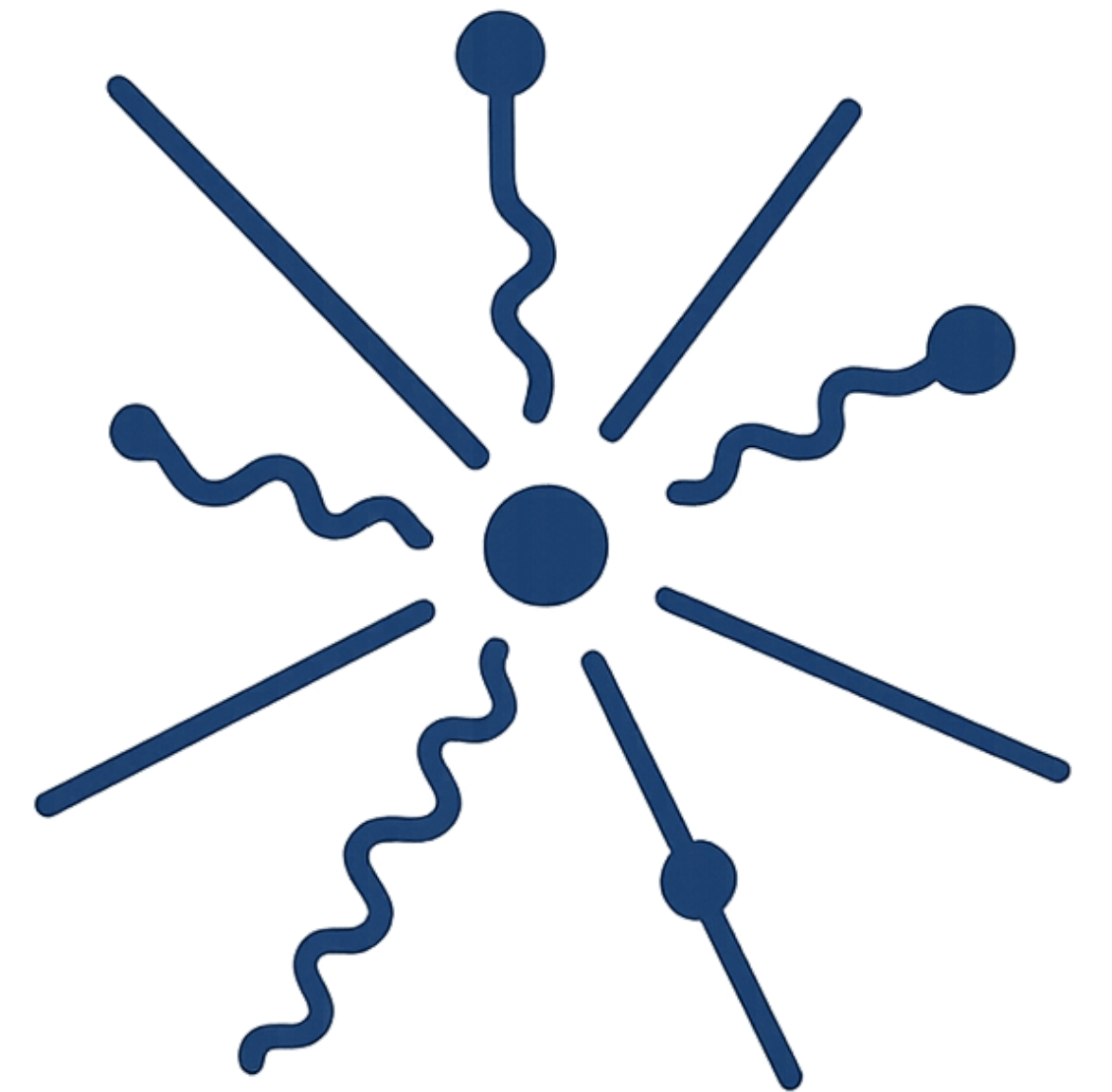
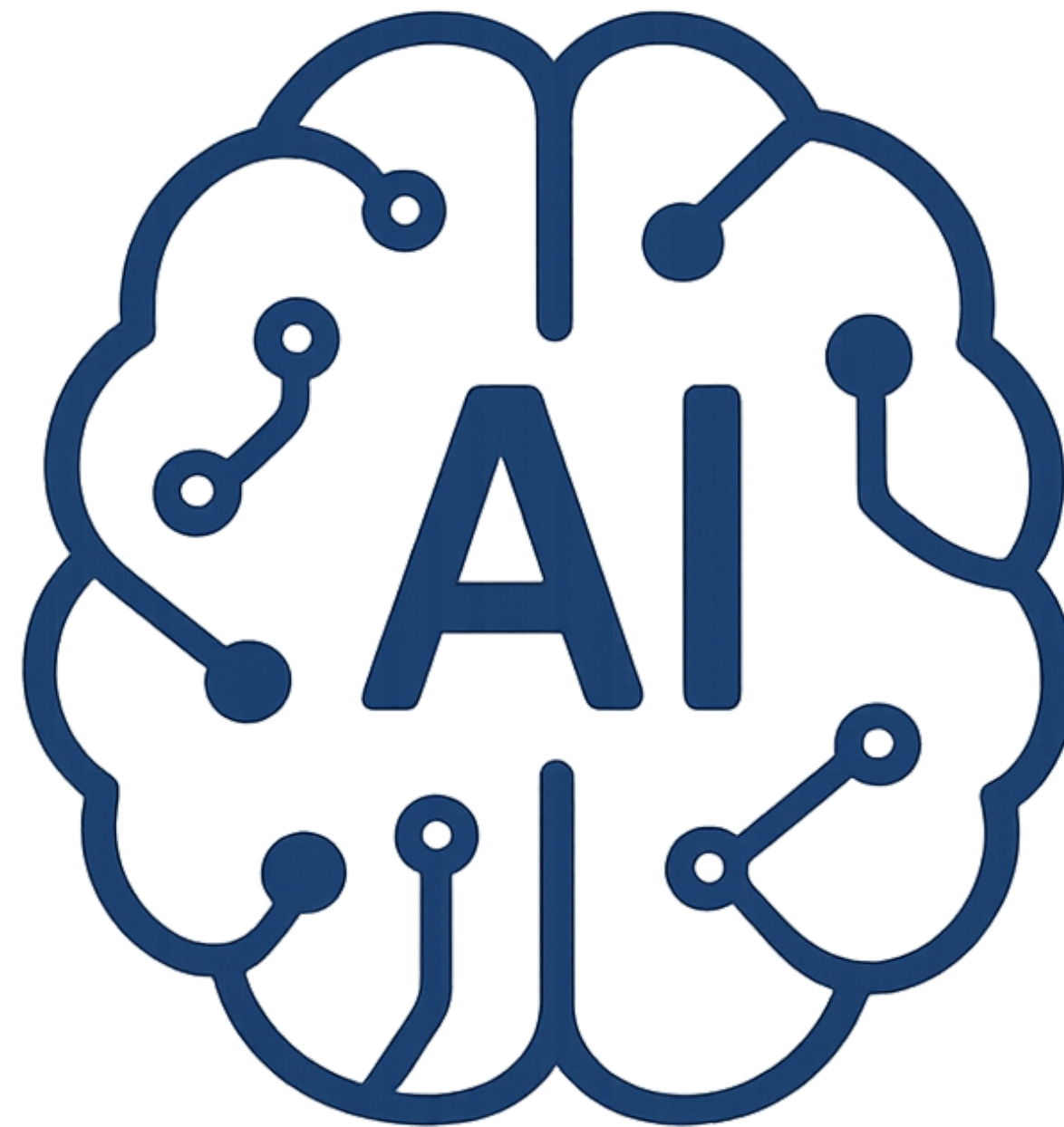
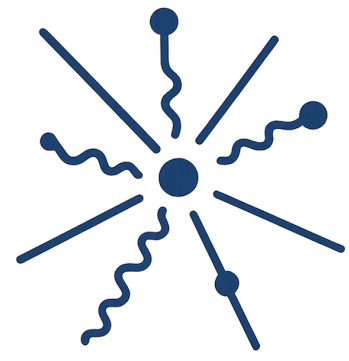
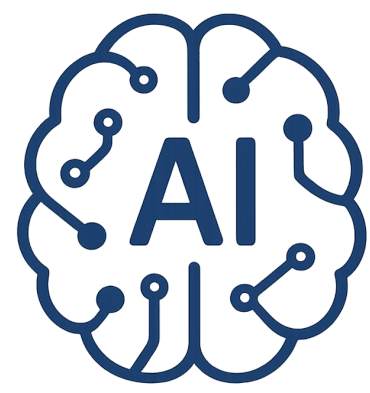


Introduction to

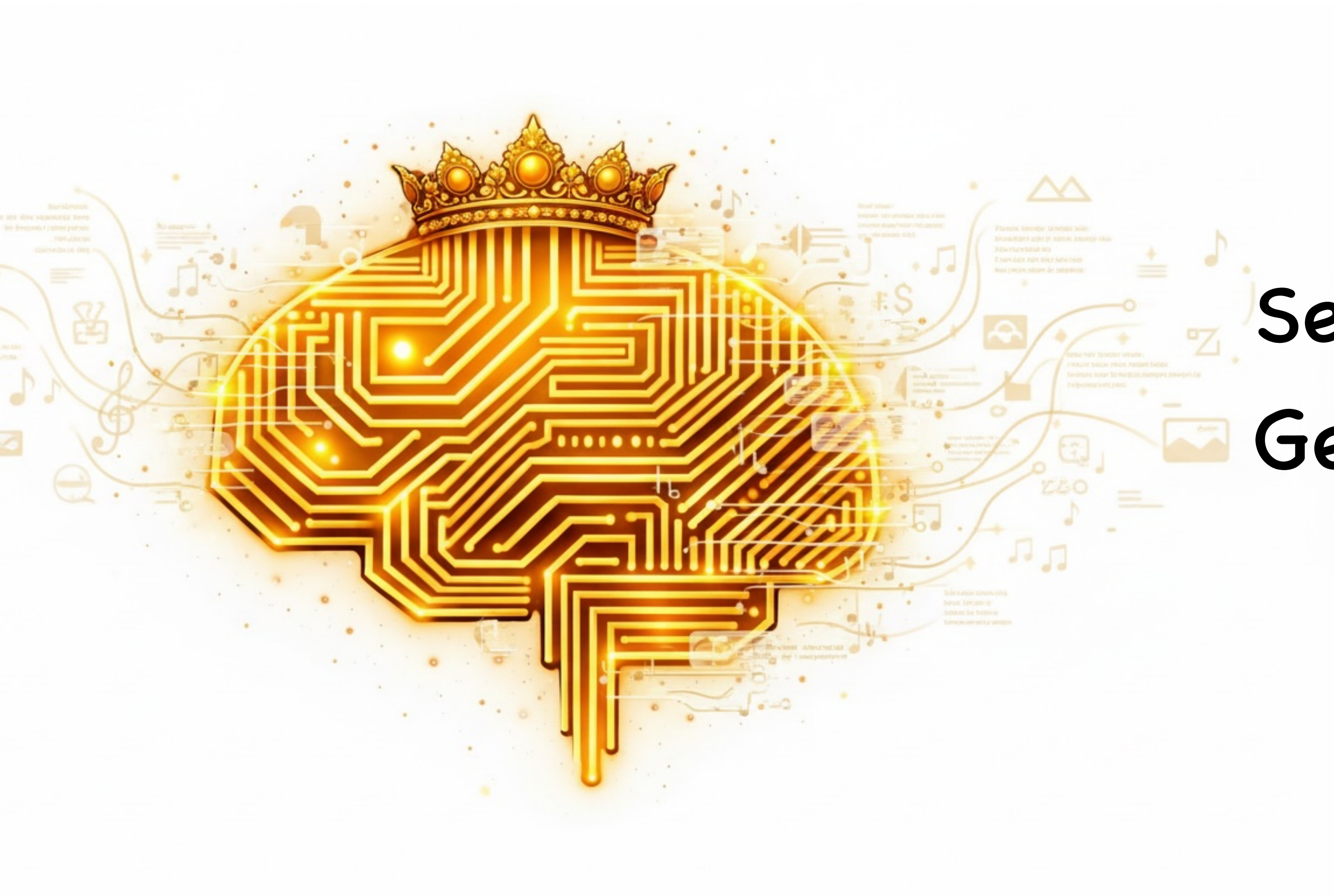
AI-Driven HEP

S. A. Fard - School of Physics (IPM)



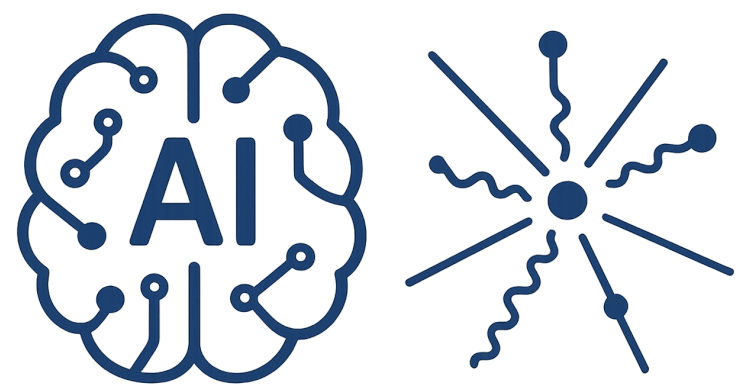


AI-Driven HEP 24



Session 24

Generative AI III



AI-Driven HEP 24

Reference

- [1] Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Springer Nature, 2023.
- [2] Simon J.D. Prince, Understanding Deep Learning, The MIT Press (2025), <https://udlbook.github.io/udlbook/>
- [3] Solar neutrino measurements using the full data period of Super-Kamiokande-IV, <https://arxiv.org/abs/2312.12907>

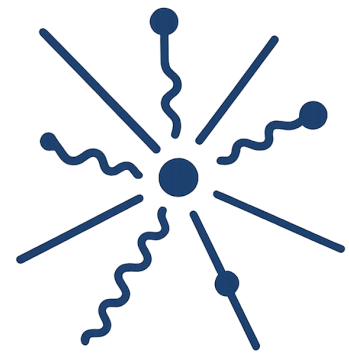
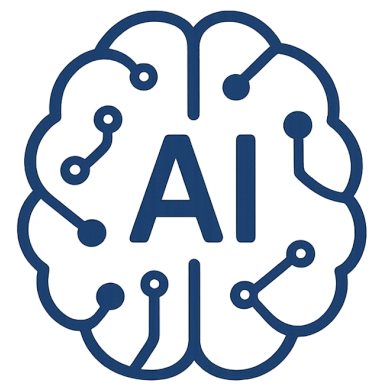
<https://physics.ipm.ac.ir/~ansarifard/>





Scientific Inference

Likelihood $p(D|H)$



Scientific Inference

Likelihood $p(D | H)$

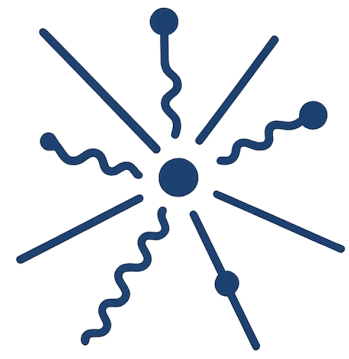
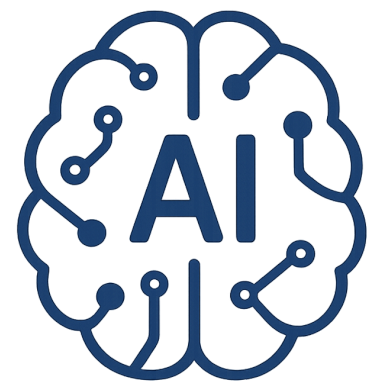
Kind of Data

Prediction

Observation

Measurement

$$p(D_p | H_p) \quad p(D_o | H_o) \quad p(D_m | H_m)$$



Scientific Inference

Likelihood $p(D | H)$

Kind of Data

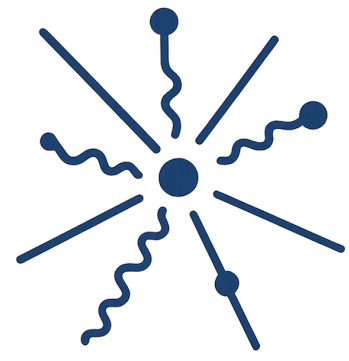
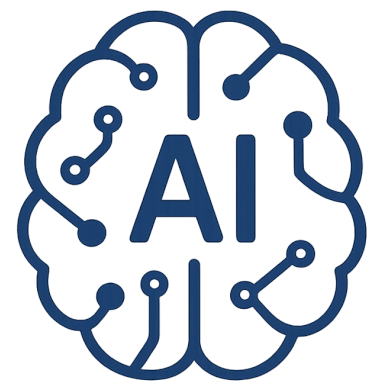
Prediction

Observation

Measurement

$$-\ln P(D | H) = (D - \mu)^\top C^{-1} (D - \mu)$$

$$p(D_p | H_p) \quad p(D_o | H_o) \quad p(D_m | H_m)$$



Scientific Inference

Likelihood $p(D | H)$

Kind of Data

Prediction

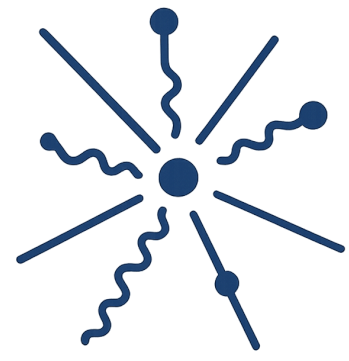
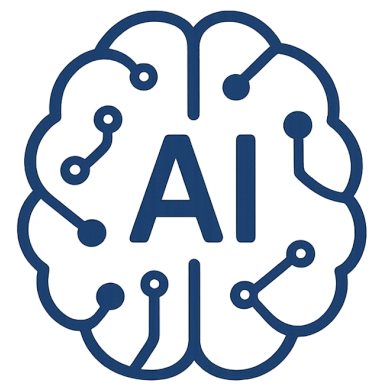
Observation

Measurement

$$p(D_p | H_p) \quad p(D_o | H_o) \quad p(D_m | H_m)$$

$$-\ln P(D | H) = (D - \mu)^\top C^{-1} (D - \mu)$$

Is the gaussian the only form
of uncertainty ?



Scientific Inference

Likelihood $p(D|H)$

Kind of Data

Prediction

Observation

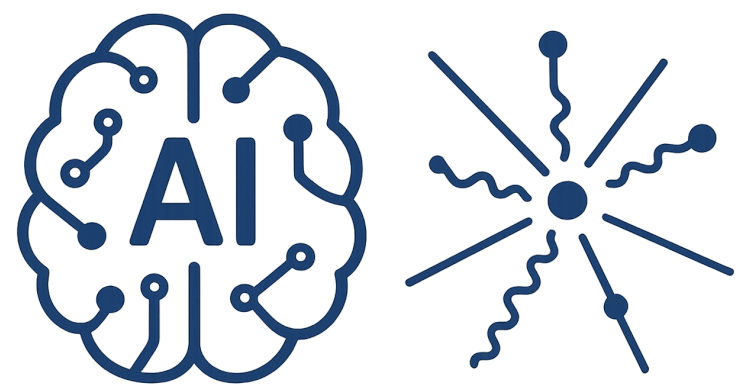
Measurement

$$p(D_p|H_p) \quad p(D_o|H_o) \quad p(D_m|H_m)$$

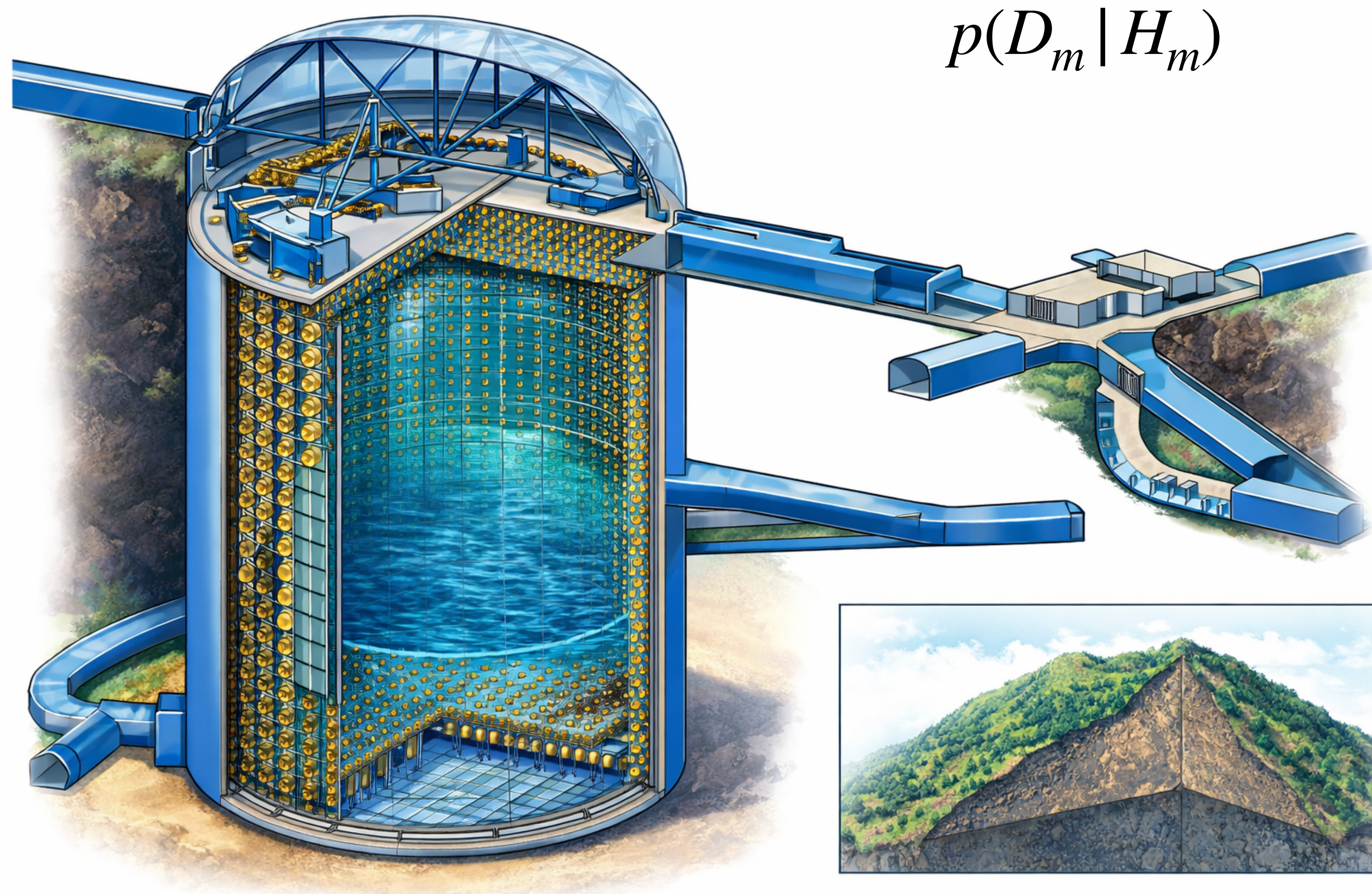
$$-\ln P(D|H) = (D - \mu)^\top C^{-1} (D - \mu)$$

Is the gaussian the only form
of uncertainty ?

How we guess the correct form of
uncertainty



SuperK as an Example



Water purification

Improve the attenuation length of Cherenkov photons

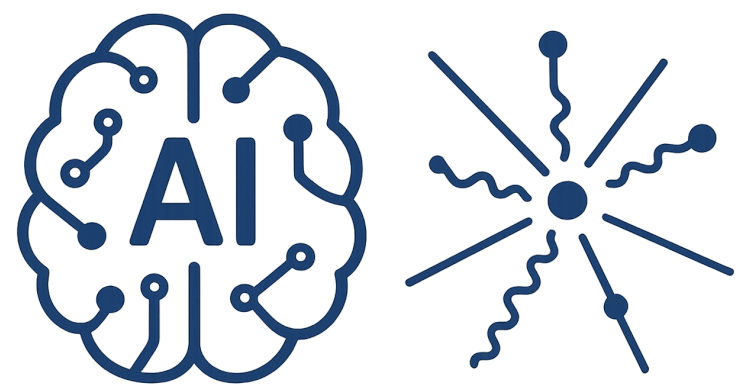
Reduce the radioactive backgrounds due to dissolved isotopes

top-bottom asymmetry

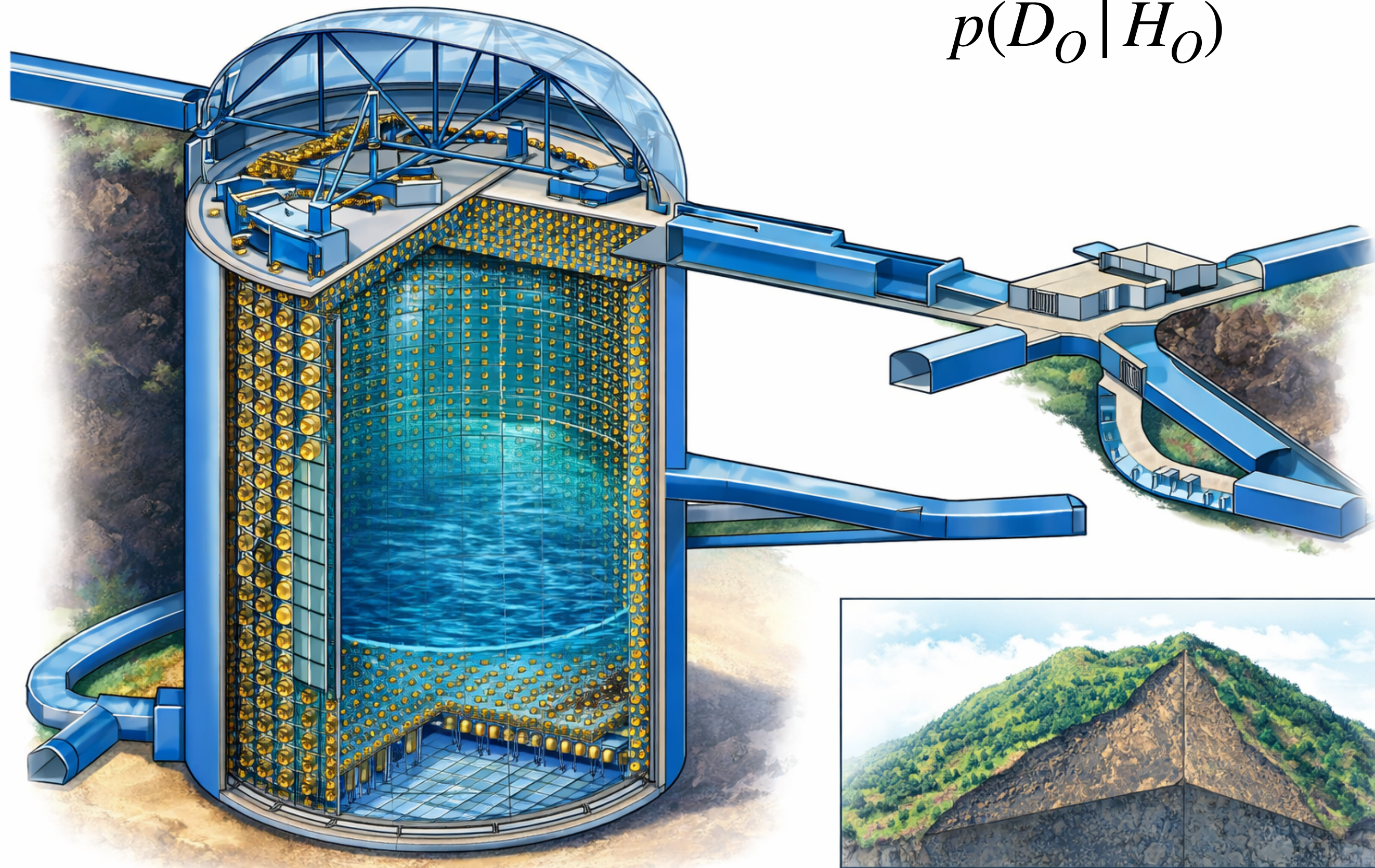
Radon radioactive contamination

PMT dark rate

PMT gain



SuperK as an Example



$$p(D_O | H_O)$$

Simulate physics

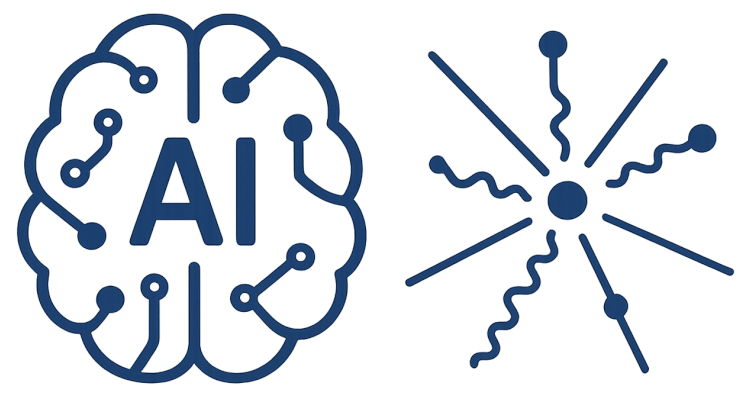
Reconstruct the response of detector

Run selection

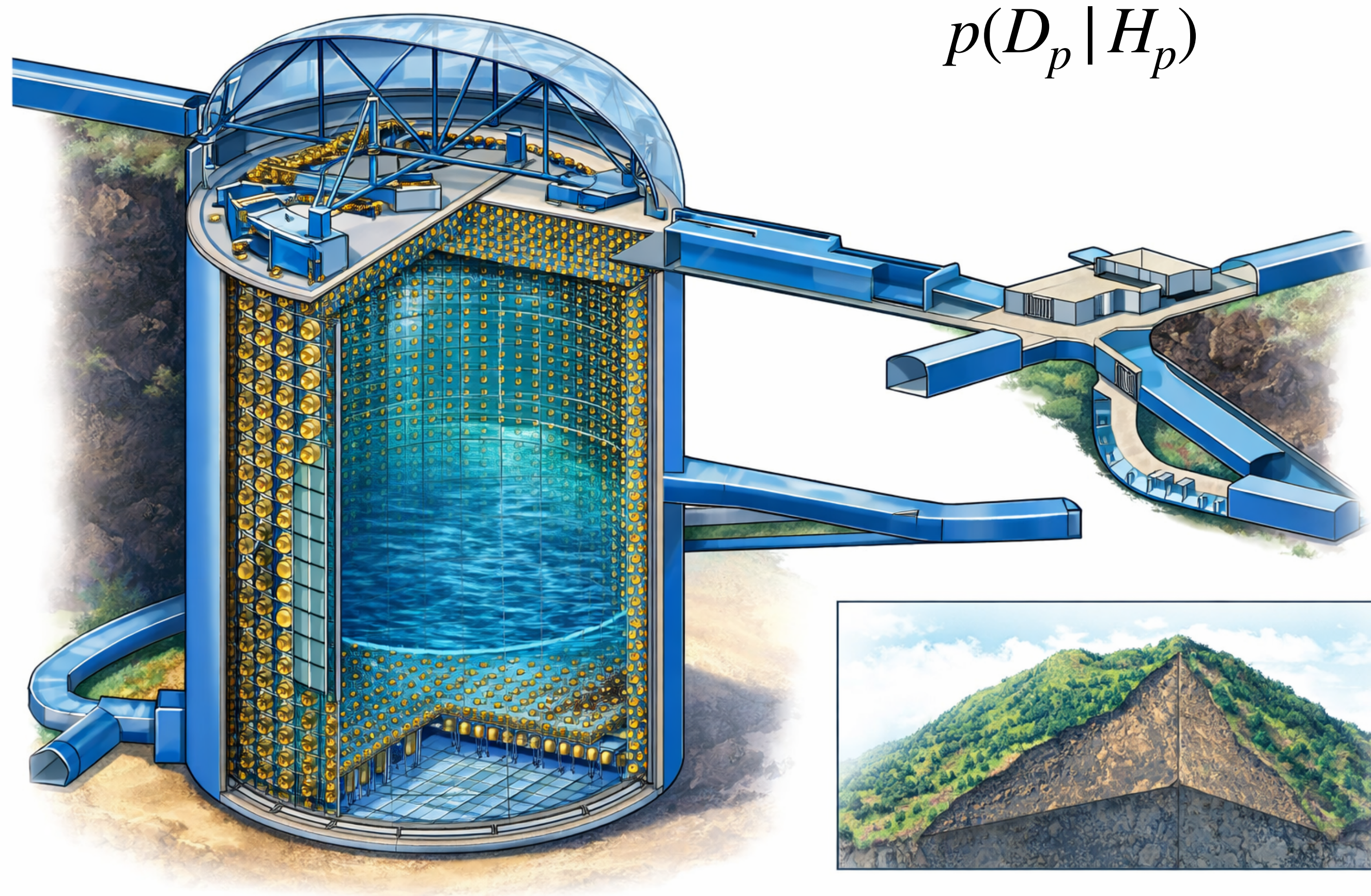
Trigger Algorithm

Data Reduction

$$\mathcal{L} = e^{-(\sum_i B_i + S)} \prod_{i=1}^{N_{\text{bin}}} \prod_{j=1}^{n_i} (B_i \cdot b_{ij} + S \cdot Y_i \cdot s_{ij})$$

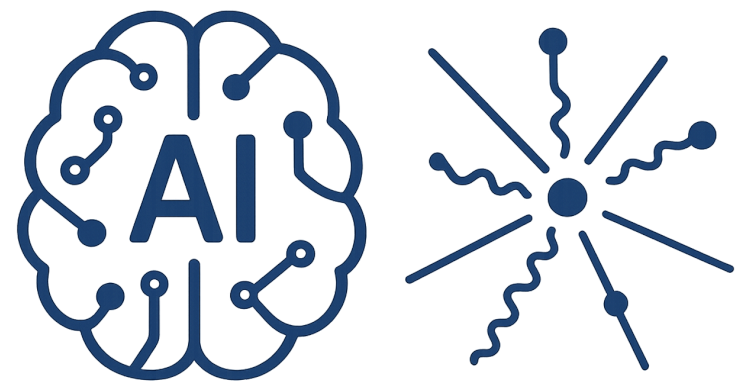


SuperK as an Example



$$p(D_p | H_p)$$

$$N_{\text{exp}} = \frac{N_{\text{MC,survival}}}{N_{\text{MC,generated}}} \times T_{\text{Live}} \times R_{\text{exp}}$$



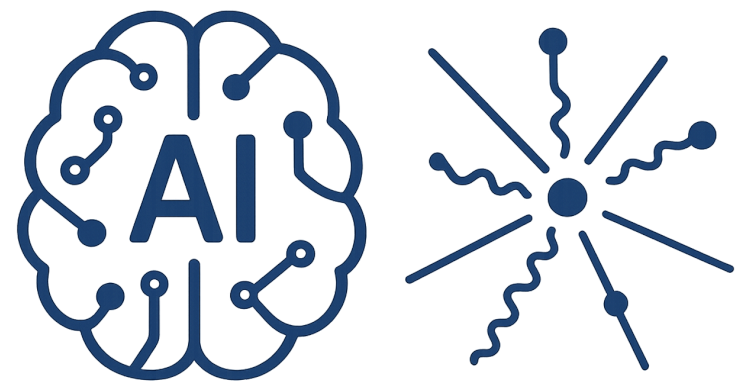
Exact and Efficient

The free parameters of a physical theory and the experimental nuisance parameters can be treated as latent variables in a generative model of the experiment.

$$p(x, \theta, \eta, z) = p(x \mid z, \theta, \eta) p(z \mid \theta) p(\theta) p(\eta)$$

$$p(x) = \int p(x \mid z) p(z) dz$$

Intractable : There is no exact, efficient algorithm to compute Likelihood



Four Approaches to Generative Modelling

Generative Adversarial Networks (GAN)

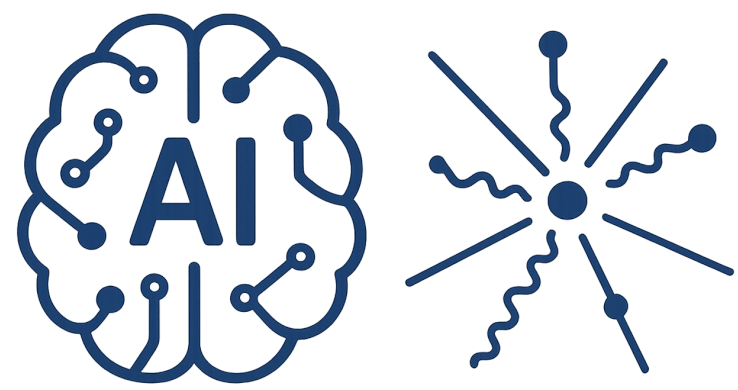
Latent Space with lower dimensions – No likelihood function – Straightforward for samples generating

Normalizing Flows

Latent and data space equal dimensions – Likelihood function – efficient sampling

Variational AutoEncoders

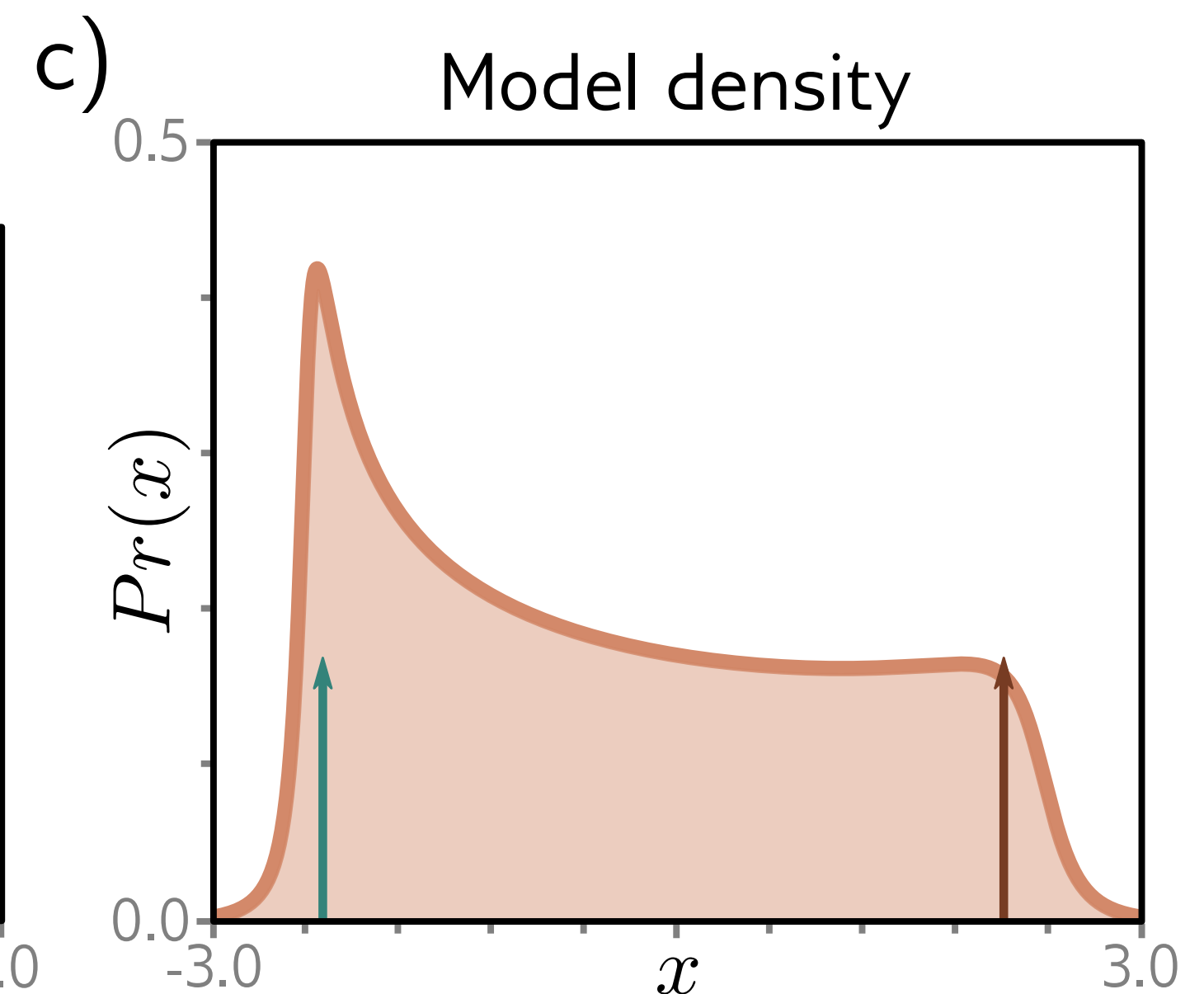
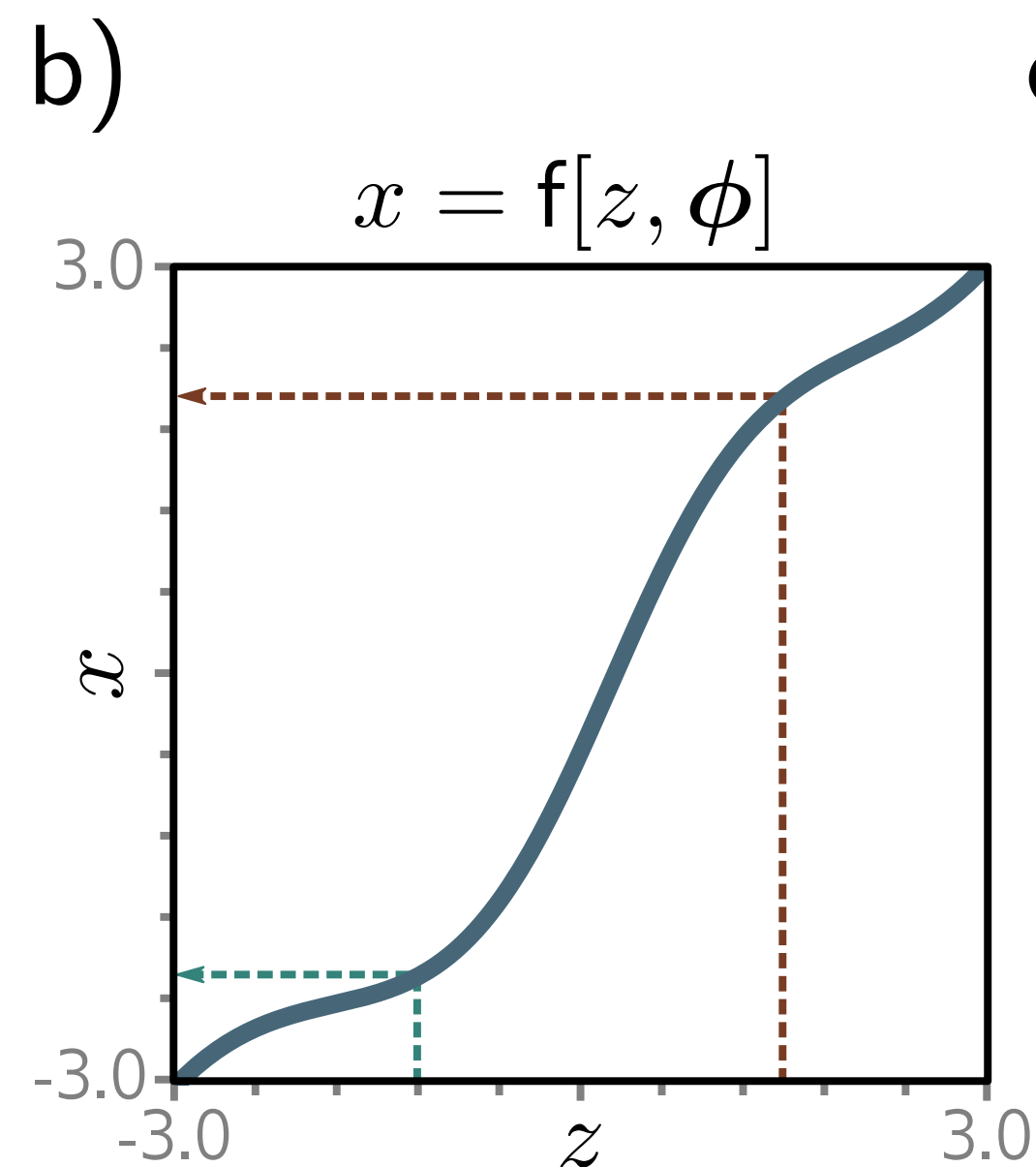
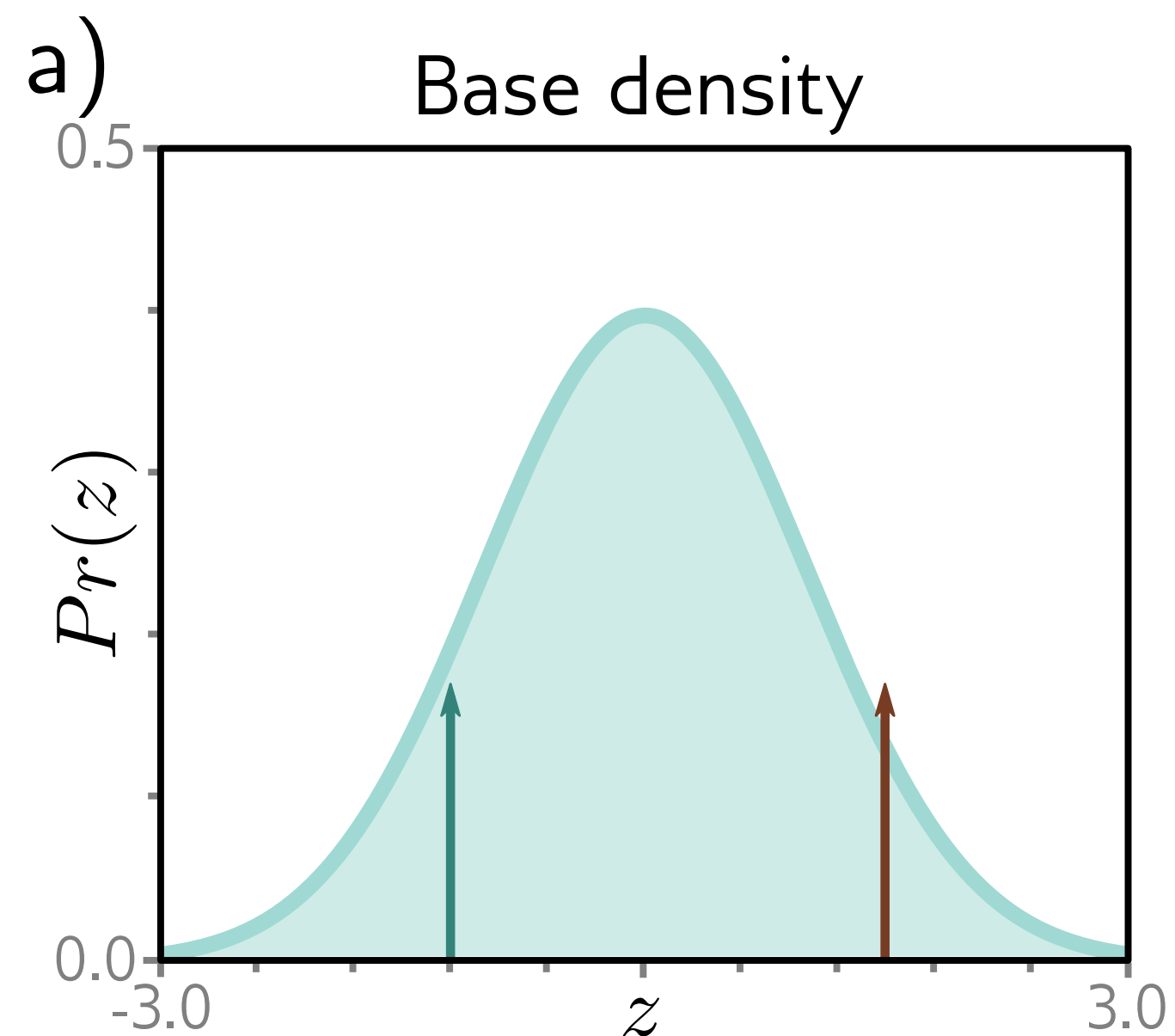
Diffusion Models

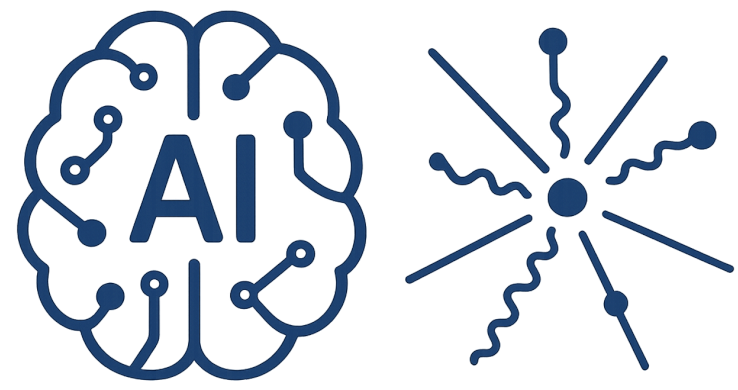


Normalizing Flows

Fit a probability distribution to training data

$$Pr(x|\phi) = \left| \frac{\partial f[z, \phi]}{\partial z} \right|^{-1} \cdot Pr(z),$$





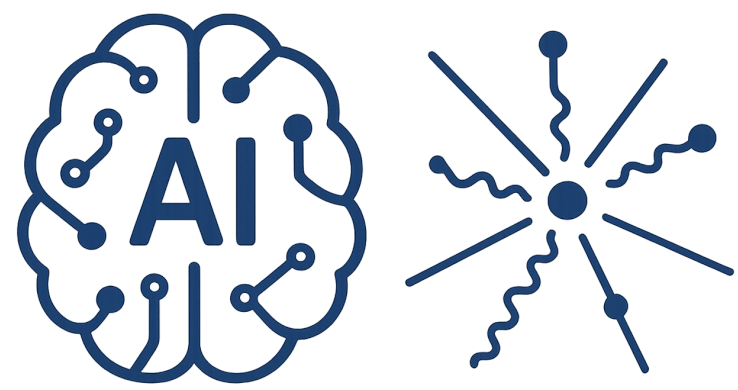
Normalizing Flows

Draw Sample : Forward Mapping $x = f[z, \phi]$

Measure the likelihood : Inverse Mapping $z = f^{-1}[x, \phi]$

$$\mathbf{x} = \mathbf{f}[\mathbf{z}, \phi] = \mathbf{f}_K \left[\mathbf{f}_{K-1} \left[\dots \mathbf{f}_2 \left[\mathbf{f}_1[\mathbf{z}, \phi_1], \phi_2 \right], \dots, \phi_{K-1} \right], \phi_K \right].$$

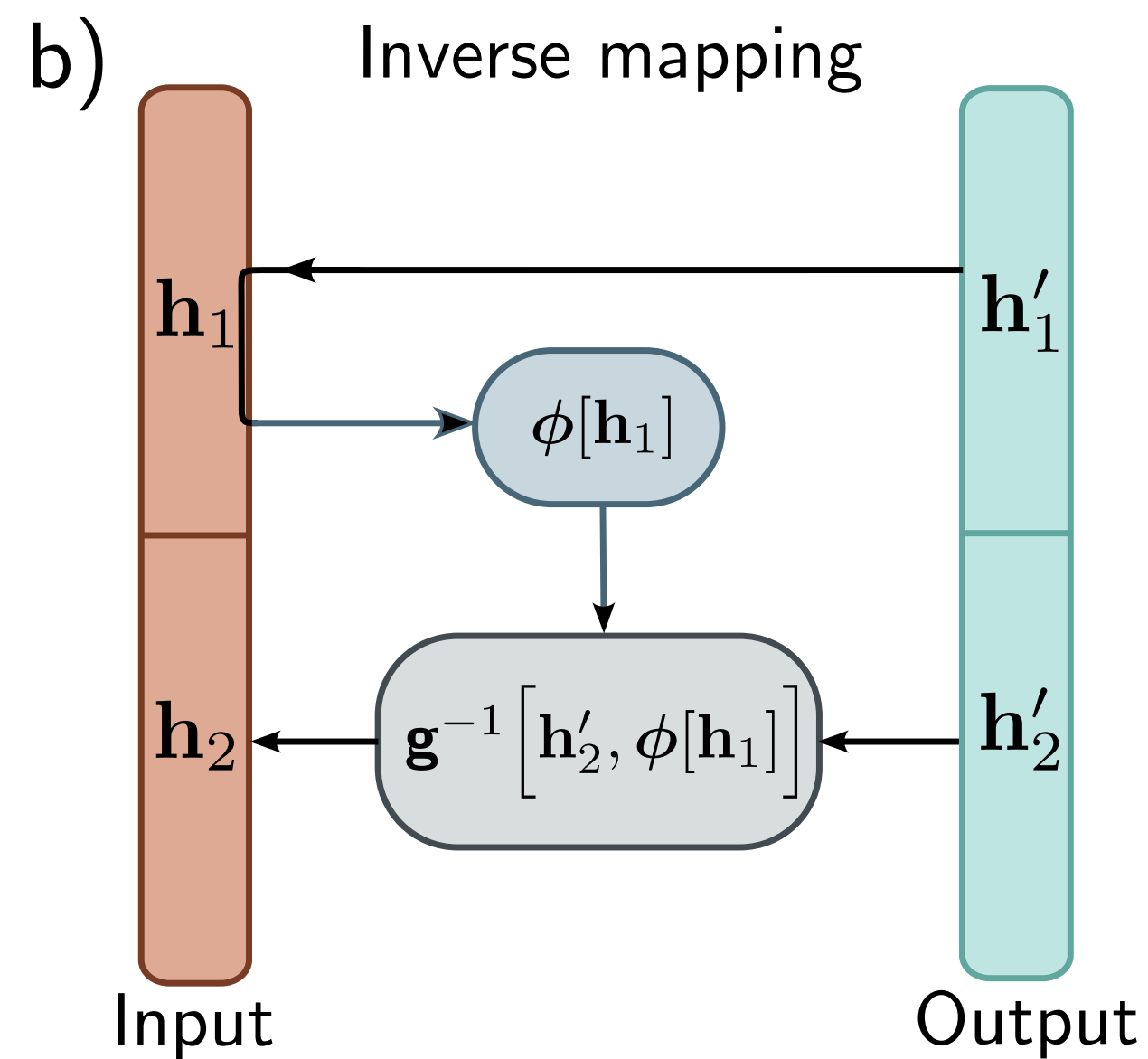
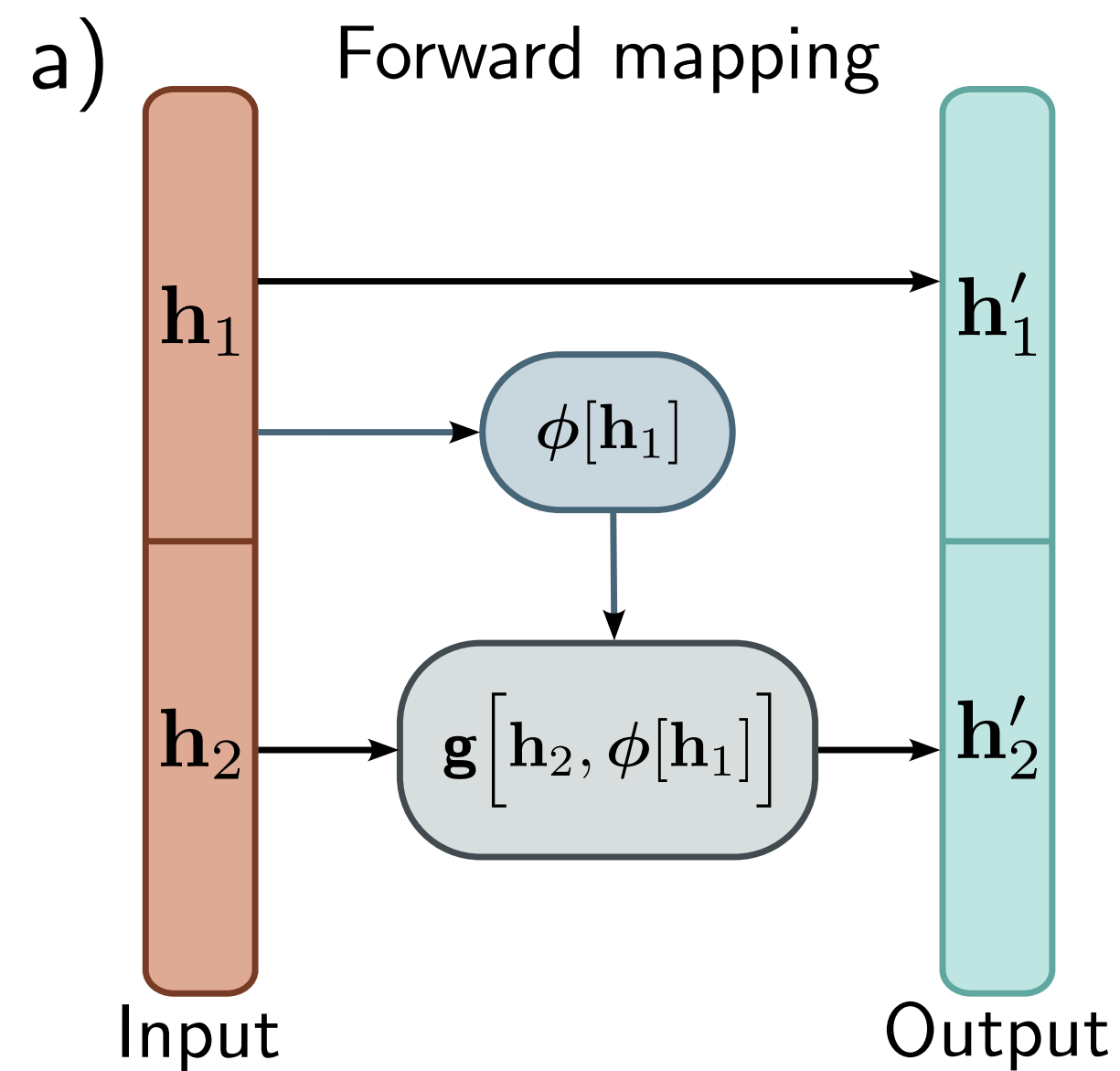
$$\mathbf{z} = \mathbf{f}^{-1}[\mathbf{x}, \phi] = \mathbf{f}_1^{-1} \left[\mathbf{f}_2^{-1} \left[\dots \mathbf{f}_{K-1}^{-1} \left[\mathbf{f}_K^{-1}[\mathbf{x}, \phi_K], \phi_{K-1} \right], \dots, \phi_2 \right], \phi_1 \right].$$



Coupling Flows

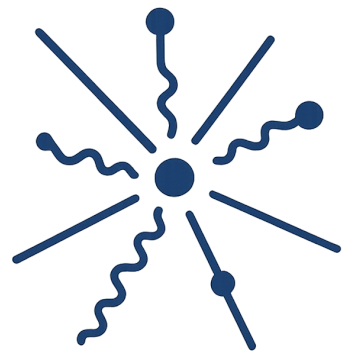
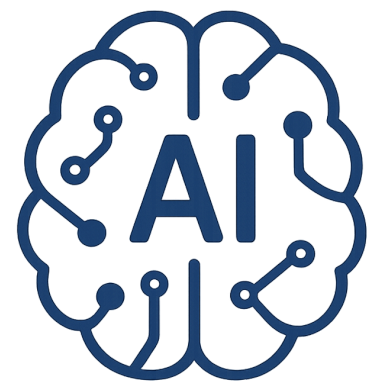
Divide the input into two parts

The Jacobian matrix will be lower triangular



ϕ Is a general non-linear function

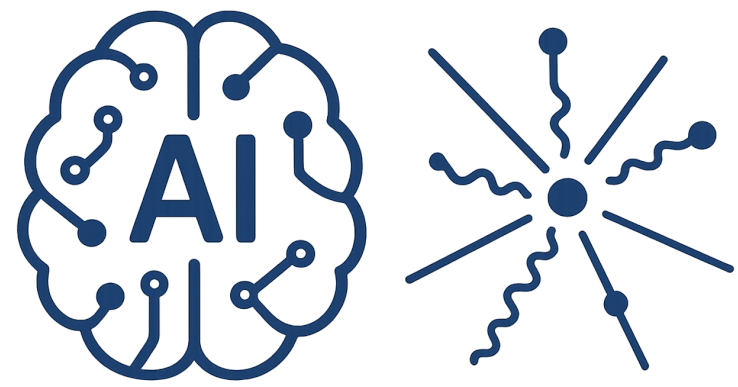
g is elementwise



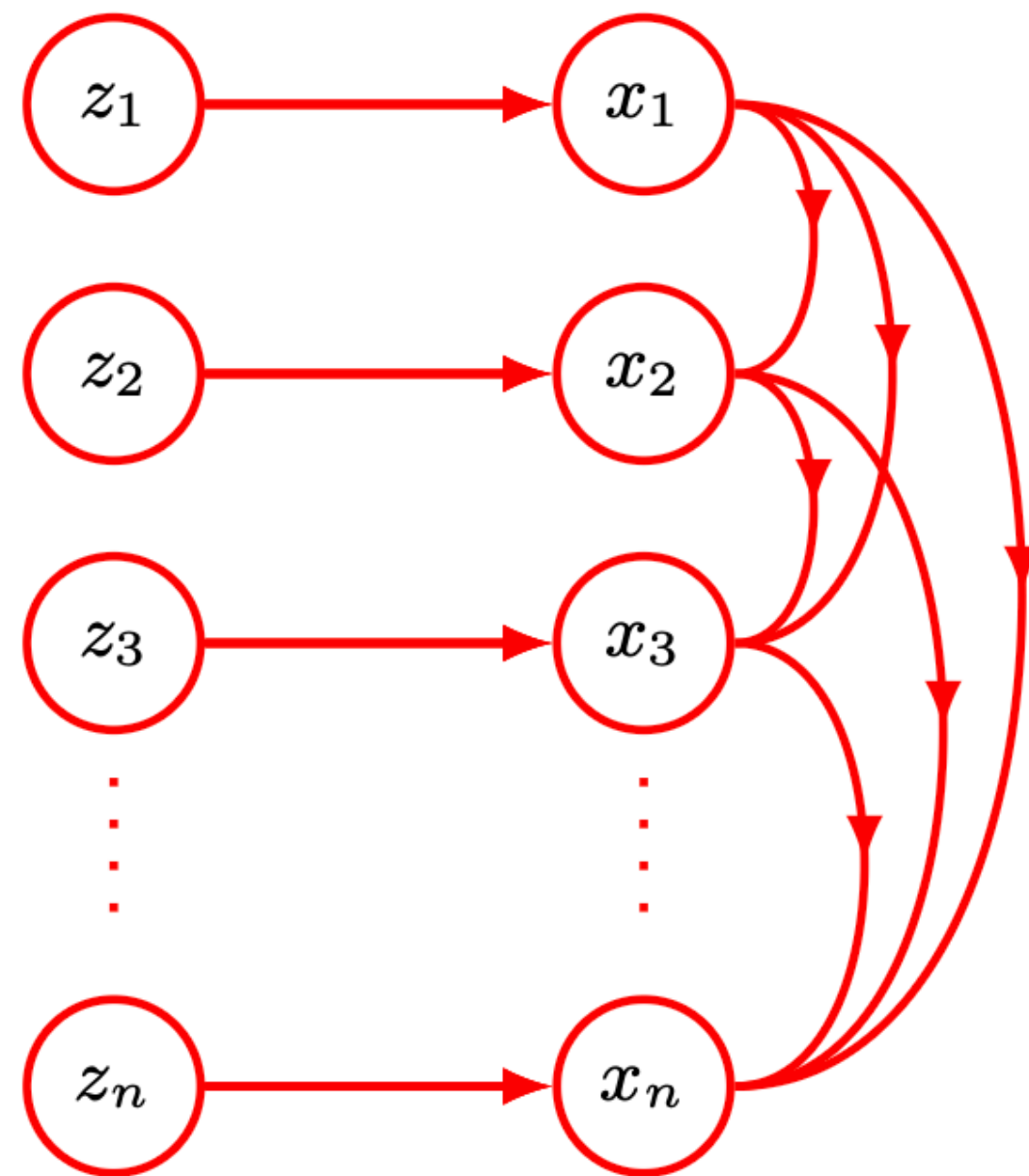
Autoregressive Flows

Generates data sequentially, where each dimension is a simple distribution conditioned on previous ones

$$p(x_1, \dots, x_D) = \prod_{i=1}^D p(x_i | \mathbf{x}_{1:i-1})$$



Autoregressive Flows

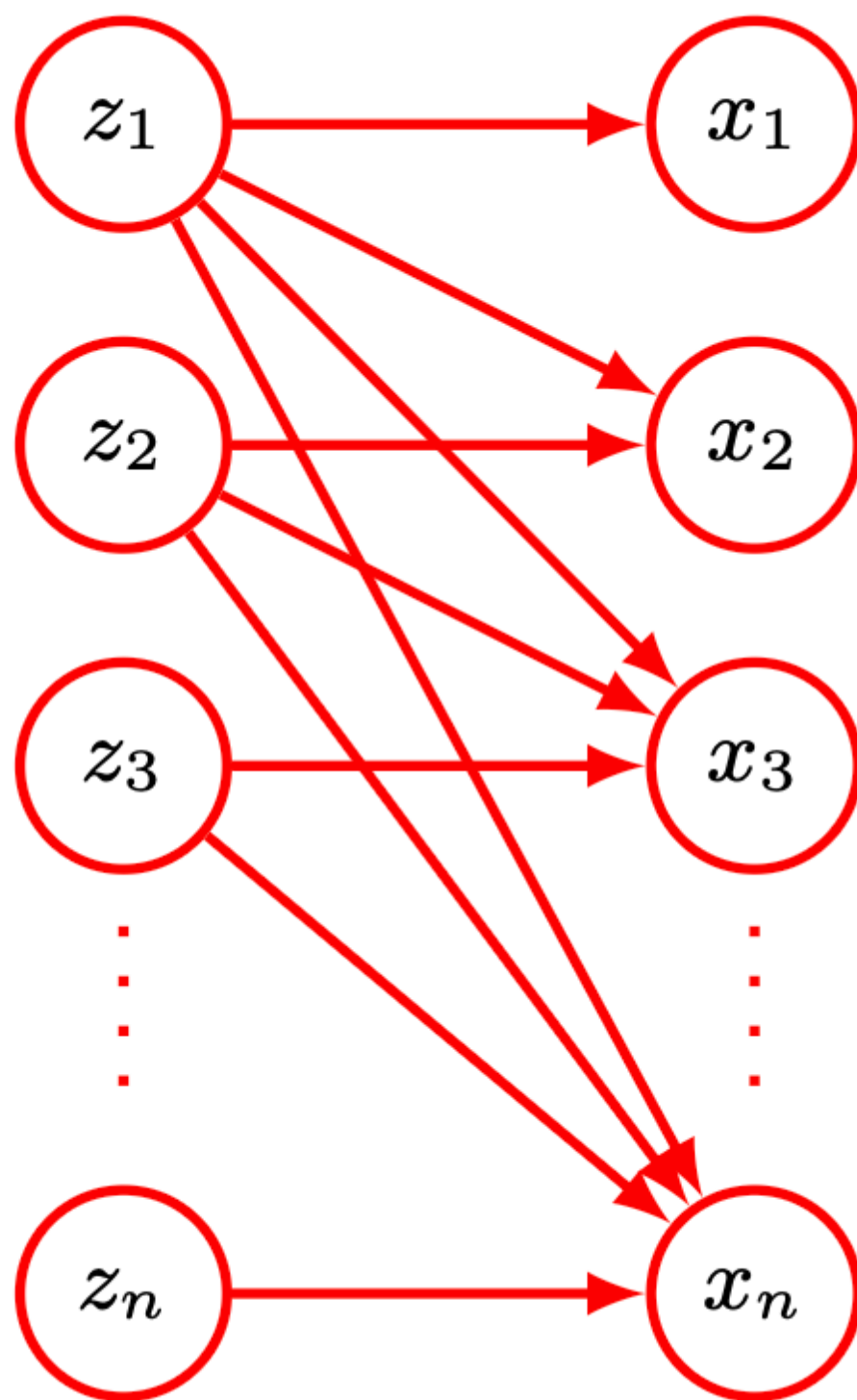


Fast for Likelihood estimation (parallel for z_i)

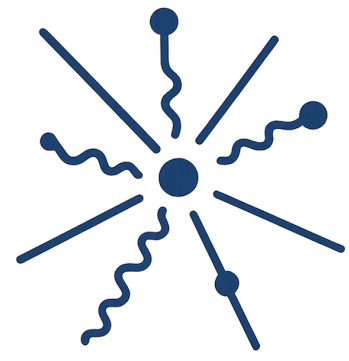
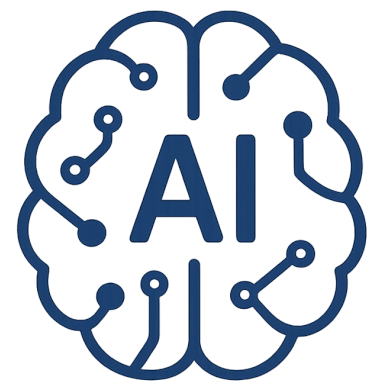
$$z_i = h^{-1}(x_i, \mathbf{g}_i(\mathbf{x}_{1:i-1}, \mathbf{w}_i))$$

Inverse Autoregressive Flows

Efficient Sampling (parallel for x_i)



$$x_i = h(z_i, \tilde{\mathbf{g}}_i(\mathbf{z}_{1:i-1}, \mathbf{w}_i))$$



Auto-Associative Neural Network

Deterministic Autoencoders

$$x \rightarrow z \rightarrow y$$

Stochastic Autoencoders

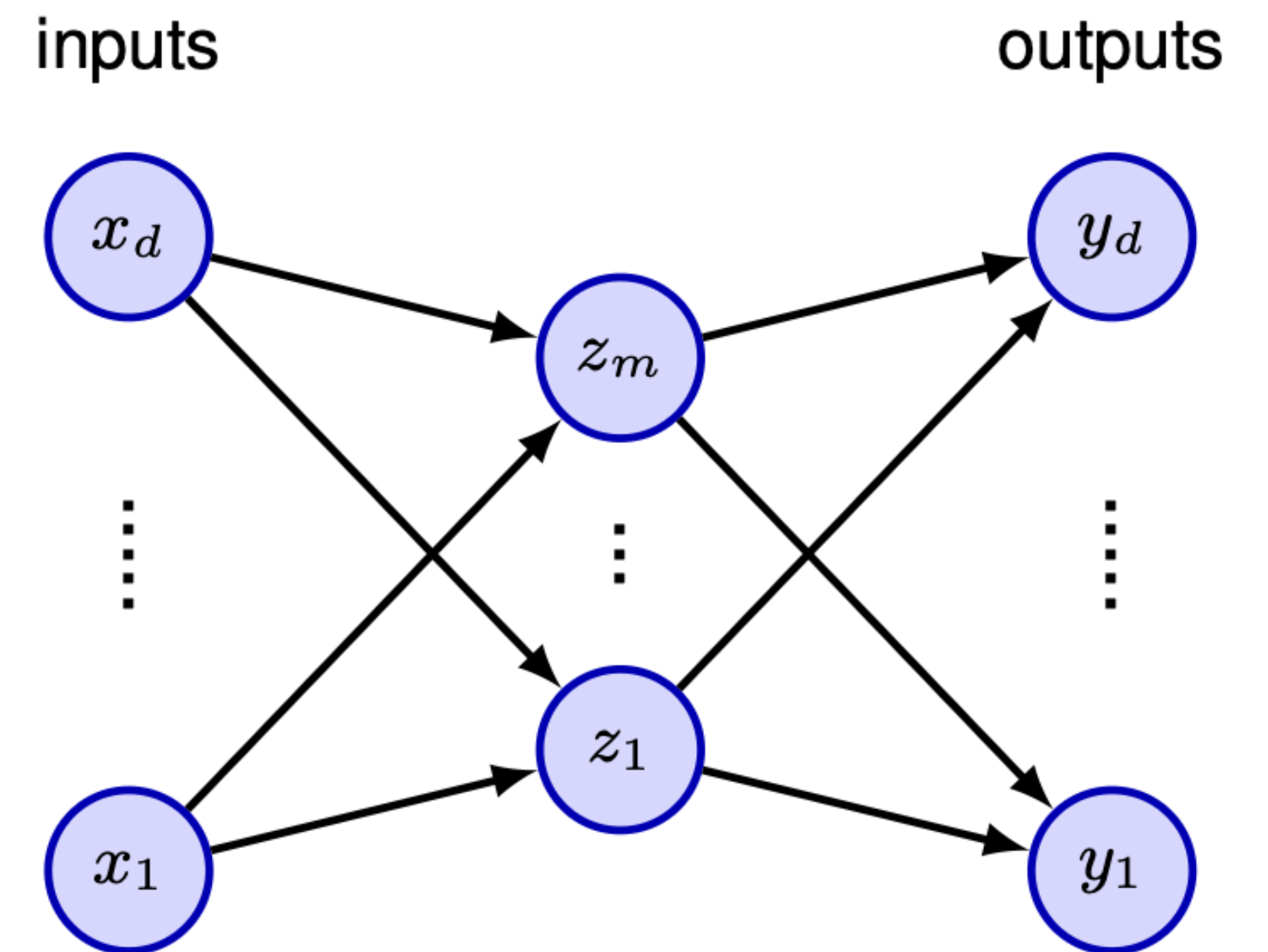
$$p(z|x), \quad p(y|z)$$

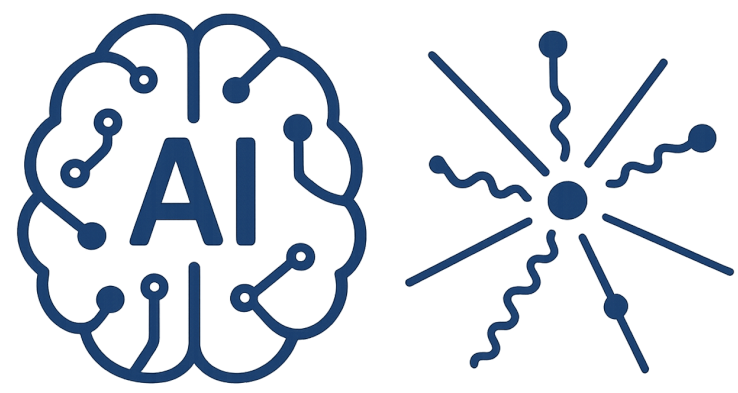
Linear Autoencoders

hidden units with linear activation functions

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N \|\mathbf{y}(\mathbf{x}_n, \mathbf{w}) - \mathbf{x}_n\|^2$$

Vectors of weights form a basis set that spans the principal subspace



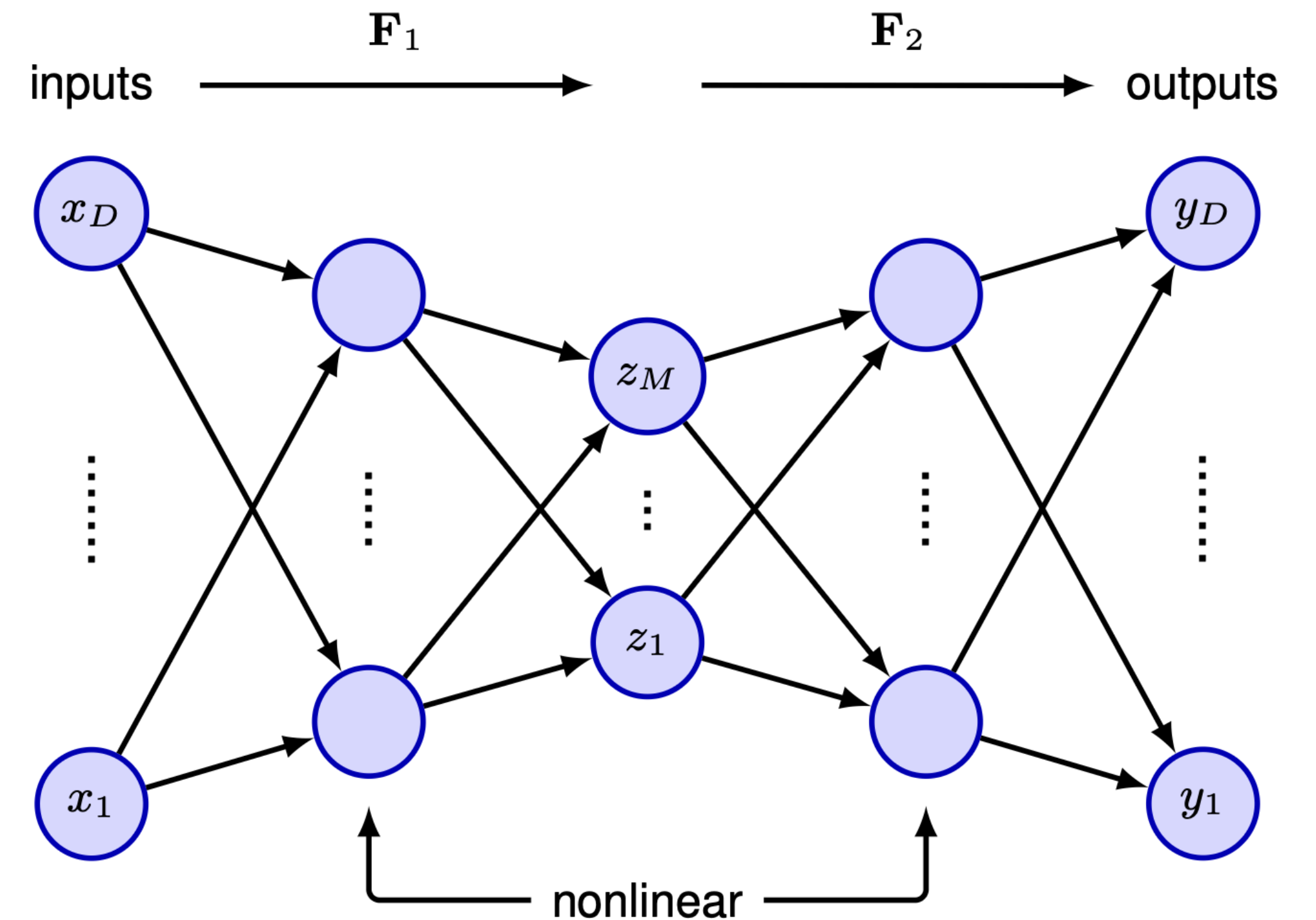


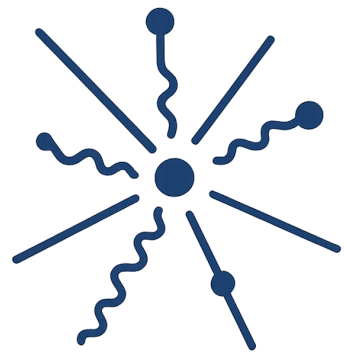
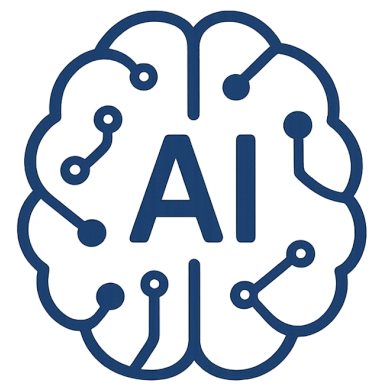
Deep Autoencoders

Similar to nonlinear form of PCA

Computationally intensive optimization

There is a risk of finding local minimum





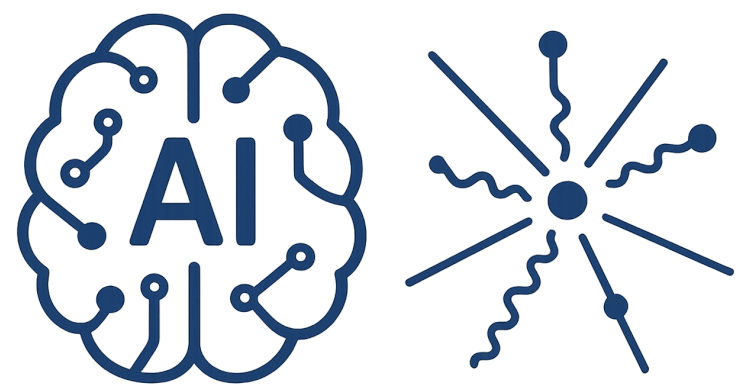
Variational Autoencoders

Goal is to maximize the likelihood function

$$p(\mathbf{x}|\mathbf{w}) = \int p(\mathbf{x}|\mathbf{z}, \mathbf{w})p(\mathbf{z}) \mathrm{d}\mathbf{z},$$

Where $p(x|z, w)$ is defined by a deep neural network

VAE approximate likelihood to train the model



Evidence Lower Bound

Starting with

$$p(\mathbf{X}|\boldsymbol{\theta}) = \sum_{\mathbf{Z}} p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta})$$

Rewrite

$$\ln p(\mathbf{X}|\boldsymbol{\theta}) = \mathcal{L}(q, \boldsymbol{\theta}) + \text{KL}(q||p)$$

Make use of product rule

$$\ln p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta}) = \ln p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta}) + \ln p(\mathbf{X}|\boldsymbol{\theta}),$$

Where

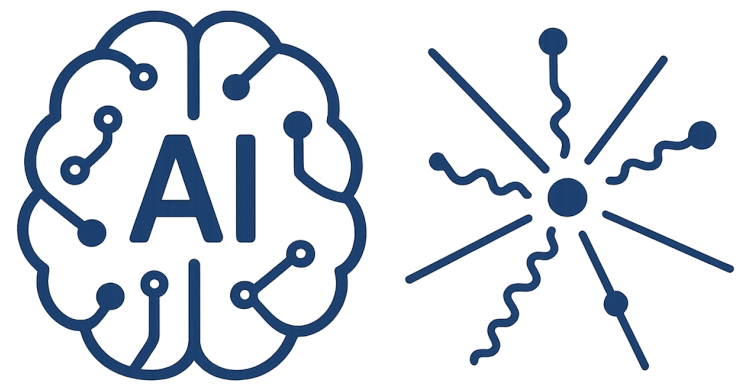
$$\mathcal{L}(q, \boldsymbol{\theta}) = \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{X}, \mathbf{Z}|\boldsymbol{\theta})}{q(\mathbf{Z})} \right\}$$

$$\text{KL}(q||p) = - \sum_{\mathbf{Z}} q(\mathbf{Z}) \ln \left\{ \frac{p(\mathbf{Z}|\mathbf{X}, \boldsymbol{\theta})}{q(\mathbf{Z})} \right\}$$

$$\text{KL}(q||p) \geq 0,$$

$$\log t \leq t - 1 \quad \text{for all } t > 0,$$

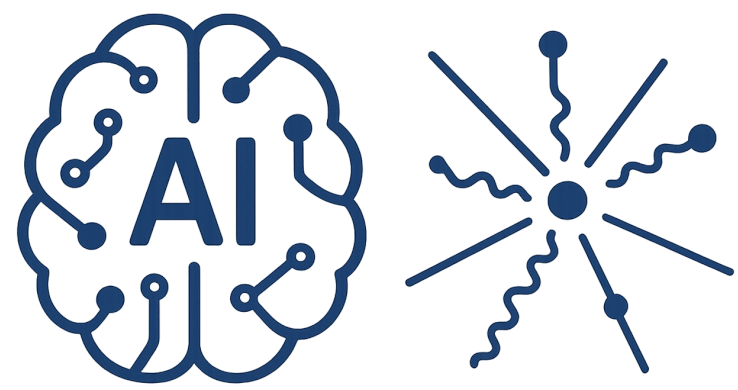
Kullback–Leibler divergence



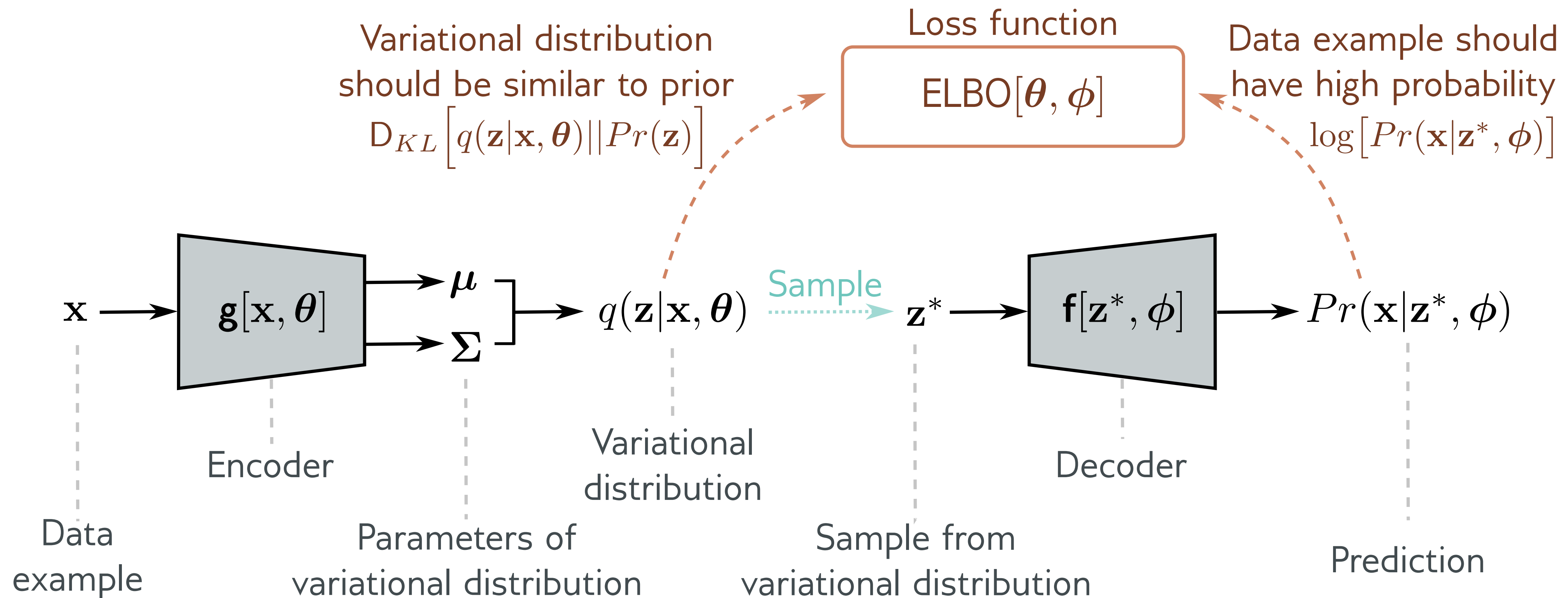
Variational approximation

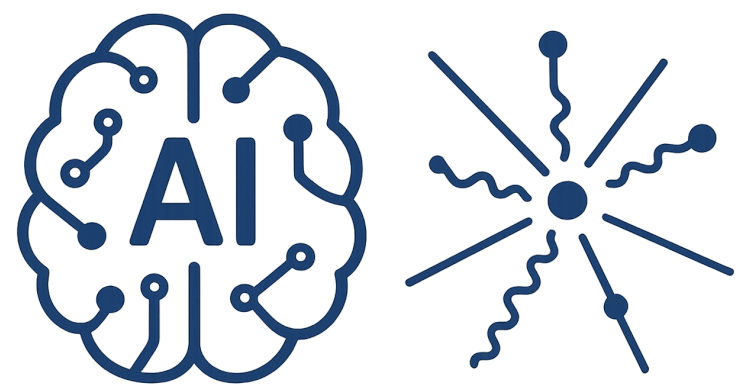
$$q(\mathbf{z}|\mathbf{x}, \phi) = \prod_{j=1}^M \mathcal{N}(z_j | \mu_j(\mathbf{x}, \phi), \sigma_j^2(\mathbf{x}, \phi)) .$$

$$\begin{aligned} \mathcal{L}_n(\mathbf{w}, \phi) &= \int q(\mathbf{z}_n | \mathbf{x}_n, \phi) \ln \left\{ \frac{p(\mathbf{x}_n | \mathbf{z}_n, \mathbf{w}) p(\mathbf{z}_n)}{q(\mathbf{z}_n | \mathbf{x}_n, \phi)} \right\} d\mathbf{z}_n \\ &= \int q(\mathbf{z}_n | \mathbf{x}_n, \phi) \ln p(\mathbf{x}_n | \mathbf{z}_n, \mathbf{w}) d\mathbf{z}_n - \text{KL}(q(\mathbf{z}_n | \mathbf{x}_n, \phi) || p(\mathbf{z}_n)) . \end{aligned}$$



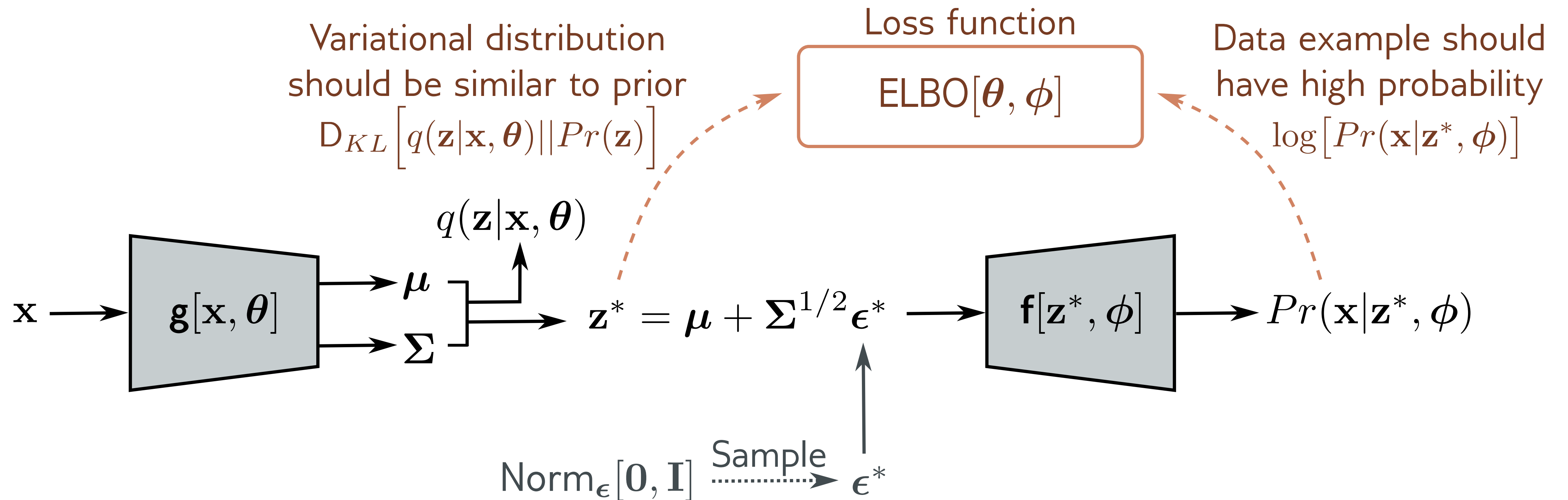
Variational Autoencoders

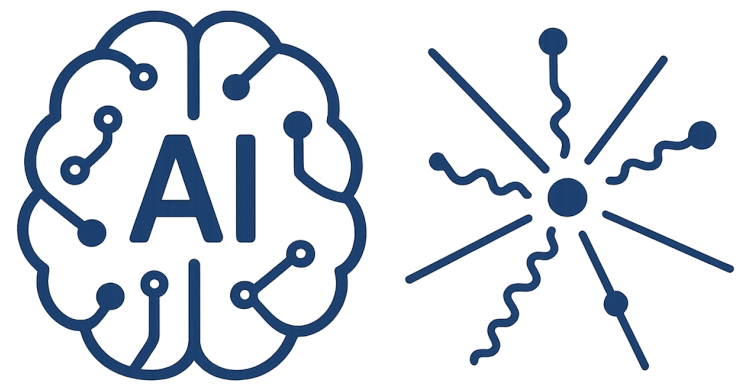




Reparameterization Trick

$$\mathcal{L} = \sum_n \left\{ \frac{1}{2} \sum_{j=1}^M \{1 + \ln \sigma_{nj}^2 - \mu_{nj}^2 - \sigma_{nj}^2\} + \frac{1}{L} \sum_{l=1}^L \ln p(\mathbf{x}_n | \mathbf{z}_n^{(l)}, \mathbf{w}) \right\}$$





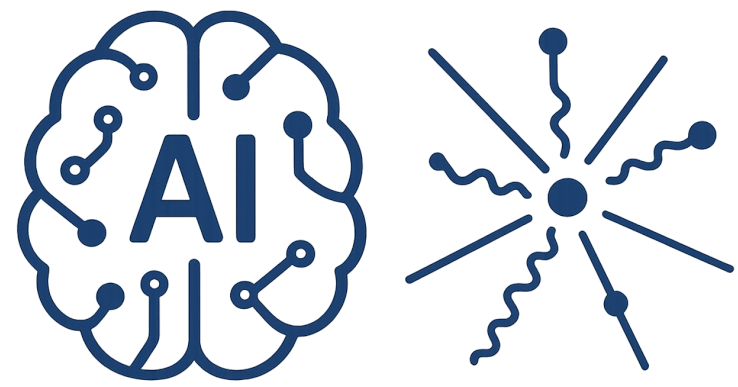
Diffusion Models

Hierarchical variational Autoencoder

the encoder distribution is fixed

$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_t$$

$$\boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{\epsilon}_t | \mathbf{0}, \mathbf{I})$$



Diffusion Models

Hierarchical variational Autoencoder

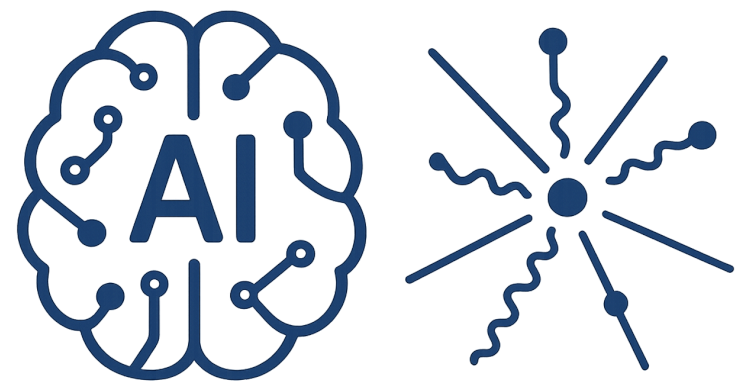
the encoder distribution is fixed

$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_t$$

$$\boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{\epsilon}_t | \mathbf{0}, \mathbf{I})$$

the generative distribution is learned

$$p(\mathbf{x} | \mathbf{w}) = \int \cdots \int p(\mathbf{x}, \mathbf{z}_1, \dots, \mathbf{z}_T | \mathbf{w}) d\mathbf{z}_1 \dots d\mathbf{z}_T$$



Diffusion Models

Hierarchical variational Autoencoder

the encoder distribution is fixed

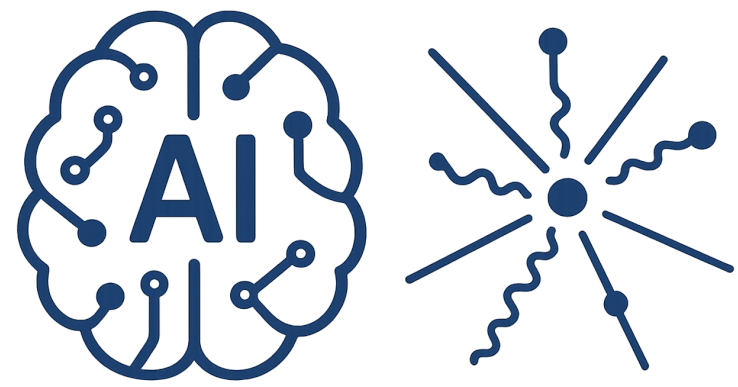
$$\mathbf{z}_t = \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_t$$

$$\boldsymbol{\epsilon}_t \sim \mathcal{N}(\boldsymbol{\epsilon}_t | \mathbf{0}, \mathbf{I})$$

the generative distribution is learned

$$p(\mathbf{x} | \mathbf{w}) = \int \cdots \int p(\mathbf{x}, \mathbf{z}_1, \dots, \mathbf{z}_T | \mathbf{w}) d\mathbf{z}_1 \dots d\mathbf{z}_T$$

$$p(\mathbf{x}, \mathbf{z}_1, \dots, \mathbf{z}_T | \mathbf{w}) = p(\mathbf{z}_T) \left\{ \prod_{t=2}^T p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w}) \right\} p(\mathbf{x} | \mathbf{z}_1, \mathbf{w})$$
$$p(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{w}) = \mathcal{N}(\mathbf{z}_{t-1} | \boldsymbol{\mu}(\mathbf{z}_t, \mathbf{w}, t), \beta_t \mathbf{I})$$
$$p(\mathbf{z}_T) = \mathcal{N}(\mathbf{z}_T | \mathbf{0}, \mathbf{I})$$



Diffusion Models

The learned network takes a noisy version of the data as input and is trained to predict how to remove the noise, step by step, to recover clean data

analogous to solving a differential equation backward in time

the diffusion model always receives the time step (or noise level) as input, encoded as an embedding, because denoising depends critically on how noisy the data is.